Modeling correlated Pauli error channel

Abstract

This note gives a Hamiltonian-free, sampling-based protocol to model *low-frequency (time-correlated)* dephasing noise in gate-based quantum circuits using a *Hidden Markov Pauli channel*. The construction produces a sequence of *conditional Pauli channels* $\{\mathcal{E}_j(\,\cdot\,|s_j)\}_{j=1}^N$ driven by a slowly varying classical hidden state s_j. Each layer remains a Pauli channel (Stim-friendly), while temporal correlations are introduced through Markovian memory of the hidden state. We provide the full problem setup, mathematical specification, parameter choices to match a desired correlation time, and an implementation recipe for Stim simulations (external sampling + Pauli injection), suitable for replication.

Contents

1 Problem setup **2**

2 Hidden Markov Pauli channel: definition **2**
2.1 Hidden state dynamics . . . 2
2.2 Conditional Pauli channel at each layer . . . 3
2.3 The correlated channel sequence . . . 3

3 Why this models low-frequency noise (without Hamiltonians) **3**
3.1 Two-state telegraph model (minimal, very useful) . . . 3
3.2 Quasi-static limit as a special case . . . 4

4 Sampling protocol (per shot) **4**
4.1 Case A: single-qubit correlated dephasing . . . 4
4.2 Case B: multi-qubit correlated dephasing, independent across qubits . . . 4
4.3 Case C: multi-qubit bursts (correlated Pauli events) . . . 4

5 Parameter choices: matching a desired "low-frequency" behavior **4**
5.1 Mean error rate . . . 5
5.2 Correlation time (two-state case) . . . 5
5.3 Practical recipe . . . 5

6 How to use this with Stim **5**
6.1 Key limitation and workaround . . . 5
6.2 Layered Clifford simulation pattern . . . 5
6.3 Python pseudo-code (replicable) . . . 6
6.4 Pauli-frame subtlety (multi-qubit, general Clifford circuits) . . . 7

7 Worked specification: two-state correlated dephasing channel **7**

8 Extensions (brief) **7**
8.1 From pure Z noise to full Pauli noise . . . 7
8.2 Spatial correlations . . . 8

9 Critical self-check (replication checklist) **8**

10 Summary **8**

1 Problem setup

We consider an n-qubit circuit executed in discrete *layers* (time steps) $j = 1, \ldots, N$. Let ρ_j be the density operator immediately *before* applying the ideal layer-j Clifford operation C_j. An "interleaved noise model" updates

$$\rho_{j+1} \;=\; \mathcal{E}_j\Big(C_j \rho_j C_j^\dagger\Big), \qquad j = 1, \ldots, N, \tag{1}$$

where $\mathcal{E}_j$ is a completely-positive trace-preserving (CPTP) noise channel applied after layer j.

Goal. Build $\{\mathcal{E}_j\}_{j=1}^N$ such that:

G1. **Pauli (Stim-compatible):** each $\mathcal{E}_j$ is a Pauli channel, ideally a mixture of Pauli errors applied after the layer.

G2. **Low-frequency temporal correlations:** errors are not i.i.d.; the noise has memory across layers (e.g. exponential correlation time $\tau_c \gg$ layer duration).

G3. **Hamiltonian-free:** avoid simulating continuous-time dynamics; instead sample a correlated discrete process that induces the desired correlations.

G4. **Simulation-ready:** provide a protocol to generate per-shot correlated error sequences and inject them into Stim simulations.

Scope. We focus on temporally correlated dephasing (Pauli-Z) noise because it is a dominant low-frequency noise mechanism. The construction extends straightforwardly to multi-Pauli noise ($X/Y/Z$ mixtures) and to spatially correlated variants (shared hidden states), which we briefly comment on later.

2 Hidden Markov Pauli channel: definition

The key idea is to introduce a slowly varying *classical hidden state* s_j that models the "noise environment" at layer j. Conditioned on s_j, the layer noise is a standard Pauli channel. Temporal correlations arise because $\{s_j\}$ is correlated.

2.1 Hidden state dynamics

Let the hidden state take values in a finite set

$$s_j \in \{1, 2, \ldots, K\}.$$

We assume s_j follows a time-homogeneous Markov chain with transition matrix $T \in \mathbb{R}^{K\times K}$:

$$\Pr(s_{j+1} = b \mid s_j = a) = T_{ab}, \qquad T_{ab} \geq 0, \qquad \sum_{b=1}^{K} T_{ab} = 1. \tag{2}$$

Let π be an initial distribution for s_1 (often the stationary distribution satisfying $\pi = \pi T$).

2.2 Conditional Pauli channel at each layer

Conditioned on $s_j = a$, define a Pauli channel $\mathcal{E}^{(a)}$. For pure dephasing (single-qubit, or applied independently on each qubit) a common choice is:

$$\mathcal{E}^{(a)}(\rho) = (1 - p_a)\rho + p_a\, Z\rho Z, \qquad p_a \in [0, 1], \tag{3}$$

where p_a is the Z-flip probability in environment state a.

More generally, for an n-qubit Pauli channel one can specify a distribution over n-qubit Paulis $P \in \mathcal{P}_n$:

$$\mathcal{E}^{(a)}(\rho) = \sum_{P \in \mathcal{P}_n} q_a(P)\, P\rho P, \qquad q_a(P) \geq 0,\ \sum_P q_a(P) = 1. \tag{4}$$

For many QEC simulations, it is enough to use independent single-qubit errors (a product distribution) or correlated multi-qubit Paulis (e.g. a burst event flipping a subset).

2.3 The correlated channel sequence

The effective layer-j noise channel is

$$\mathcal{E}_j(\cdot) \;=\; \mathcal{E}^{(s_j)}(\cdot), \tag{5}$$

where s_j is sampled from the Markov chain (2). This produces a *random* channel sequence $\{\mathcal{E}_j\}_{j=1}^N$. A single Monte Carlo *shot* corresponds to one sample path $(s_1, \ldots, s_N)$ and one corresponding Pauli error realization.

Definition 1 (Hidden Markov Pauli channel)**.** *A* Hidden Markov Pauli channel *is the random process defined by* (2) *and* (4)–(5)*.*

3 Why this models low-frequency noise (without Hamiltonians)

Low-frequency noise means that the "noise strength" drifts slowly: adjacent layers see similar noise. In this model, "noise strength" is encoded by s_j. If T has large diagonal entries, then $s_{j+1} = s_j$ with high probability, i.e. the environment changes slowly.

3.1 Two-state telegraph model (minimal, very useful)

The simplest nontrivial case is $K = 2$, with states $s \in \{0, 1\}$ representing *quiet* and *noisy* regimes:

$$p_0 = p_{\text{low}}, \qquad p_1 = p_{\text{high}}. \tag{6}$$

Let the Markov chain be

$$T = \begin{pmatrix} 1-\gamma & \gamma \\ \gamma & 1-\gamma \end{pmatrix}, \qquad \gamma \in [0, 1]. \tag{7}$$

Then the expected run length in a state is $\approx 1/\gamma$ layers, giving a correlation time (in layers) of order $1/\gamma$. This yields *bursty* dephasing: once the environment is "bad", Z errors remain more likely for many subsequent layers.

Remark 1 (Correlation time calibration (rule of thumb))**.** *For* (7)*, the state autocorrelation decays like* $(1 - 2\gamma)^{|j-k|}$*. Thus a convenient layer-based correlation time is*

$$\kappa \;\approx\; \frac{-1}{\ln(1 - 2\gamma)} \;\approx\; \frac{1}{2\gamma} \quad (\gamma \ll 1).$$

If each layer corresponds to physical time τ*, then* $\tau_c \approx \kappa\,\tau$*.*

3.2 Quasi-static limit as a special case

If the noise is extremely low-frequency over the circuit duration, one may take $T = I$ (or $\gamma \to 0$ in (7)), so that s_j is constant across all layers in a shot. Then each shot experiences a fixed noise strength p_s, but shot-to-shot the strength changes. This is a standard discrete proxy for quasi-static noise and already produces strong temporal correlations.

4 Sampling protocol (per shot)

We now write the sampling procedure explicitly for the most common case: temporally correlated Z noise (possibly on multiple qubits).

4.1 Case A: single-qubit correlated dephasing

Fix K, transition matrix T, and per-state dephasing probabilities $\{p_a\}_{a=1}^{K}$.

A single shot produces a binary error indicator sequence $e_1, \ldots, e_N$ where $e_j = 1$ means apply Z after layer j:

$$s_1 \sim \pi, \quad s_{j+1} \sim T_{s_j,\cdot}, \quad e_j \sim \text{Bernoulli}(p_{s_j}). \tag{8}$$

The corresponding per-layer channel realization is

$$\mathcal{E}_j(\rho) = \begin{cases} \rho, & e_j = 0, \\ Z\rho Z, & e_j = 1. \end{cases}$$

4.2 Case B: multi-qubit correlated dephasing, independent across qubits

For n qubits, the simplest extension is to use the *same* hidden state s_j for all qubits (capturing a global drift) but independent Bernoulli draws per qubit:

$$e_j^{(q)} \sim \text{Bernoulli}(p_{s_j}), \qquad q = 1, \ldots, n. \tag{9}$$

Then apply Z on each qubit q with $e_j^{(q)} = 1$. This creates temporal correlations (from s_j) and also mild spatial correlation if s_j is shared.

4.3 Case C: multi-qubit bursts (correlated Pauli events)

To model bursts, one can define, in state a, a distribution over subsets $B \subseteq \{1, \ldots, n\}$:

$$\Pr(B_j = B \mid s_j = a) = r_a(B), \qquad \sum_B r_a(B) = 1, \tag{10}$$

and apply the multi-qubit Pauli $Z_B \equiv \prod_{q \in B} Z_q$ after layer j. This is still a Pauli channel, but can produce long error strings and strongly affect decoder performance.

5 Parameter choices: matching a desired "low-frequency" behavior

This model is deliberately modular. In practice you typically want to match: (i) the mean error rate per layer, and (ii) a correlation time.

5.1 Mean error rate

Let $\bar{p}$ be the desired marginal Z error probability per layer (averaged over long time / stationary regime). If s_j is stationary with distribution π, then

$$\mathbb{E}[p_{s_j}] = \sum_{a=1}^{K} \pi_a p_a = \bar{p}. \tag{11}$$

This is a linear constraint on $\{p_a\}$ (given π).

5.2 Correlation time (two-state case)

For $K = 2$ with (7) and stationary $\pi = (1/2, 1/2)$, the hidden-state autocorrelation is $(1 - 2\gamma)^{|j-k|}$. The error indicators e_j inherit correlations of the same functional form (up to a scaling factor depending on $p_{\text{low}}, p_{\text{high}}$). Thus choosing $\gamma \ll 1$ gives low-frequency drift; the effective correlation time is $\kappa \approx 1/(2\gamma)$ layers.

5.3 Practical recipe

A practical and transparent recipe is:

R1. Choose a small γ to set correlation length $\kappa \approx 1/(2\gamma)$ layers.

R2. Choose p_{low} and p_{high} to match the mean $\bar{p}$ and the variance of the instantaneous rate you want. A simple choice is:

$$p_{\text{low}} = \bar{p} - \Delta, \quad p_{\text{high}} = \bar{p} + \Delta,$$

clipped to $[0, 1]$, where Δ sets how "bursty" the noise is.

6 How to use this with Stim

6.1 Key limitation and workaround

Stim's built-in stochastic noise instructions (e.g. `Z_ERROR(p)`) generate i.i.d. errors *within a single sampling call*. Therefore, to simulate *temporally correlated* errors, one should:

S1. Sample the hidden-state path and the corresponding Pauli errors *externally* (in Python).

S2. Inject those Pauli errors into the simulation *between layers*.

This keeps the circuit Clifford while allowing non-i.i.d. error sequences.

6.2 Layered Clifford simulation pattern

Assume your ideal circuit can be decomposed as a product of layer circuits:

$$C = C_N C_{N-1} \cdots C_1, \tag{12}$$

where each C_j is a Stim circuit consisting of Clifford gates and (optionally) measurements/resets for that layer.

For a single shot, the noisy evolution is simulated by iterating:

$$\rho_{j+1} = \widetilde{\mathcal{E}}_j\left(C_j \rho_j C_j^\dagger\right), \tag{13}$$

where $\widetilde{\mathcal{E}}_j$ is the *sampled* Pauli error (e.g. apply Z or apply identity). This is exactly what Stim's `TableauSimulator` supports: apply Clifford layer C_j, then apply Pauli(s).

6.3 Python pseudo-code (replicable)

The following pseudo-code captures the essential logic. It assumes: (i) you have a list `layers` of Stim circuits C_j, (ii) you know which qubit(s) the Z noise applies to (or you sample subsets), (iii) you externally sample e_j (or subsets B_j).

```
import stim
import random

def sample_hidden_markov_path(N, pi, T):
    # pi: list length K, T: list of lists KxK
    K = len(pi)
    # sample s1
    r = random.random()
    cum = 0.0
    s = 0
    for a in range(K):
        cum += pi[a]
        if r <= cum:
            s = a
            break
    path = [s]
    for _ in range(N-1):
        a = path[-1]
        r = random.random()
        cum = 0.0
        b = 0
        for bb in range(K):
            cum += T[a][bb]
            if r <= cum:
                b = bb
                break
        path.append(b)
    return path

def sample_correlated_Z_errors(N, pi, T, p_by_state):
    # returns binary list e_j
    s_path = sample_hidden_markov_path(N, pi, T)
    e = []
    for j in range(N):
        p = p_by_state[s_path[j]]
        e.append(1 if random.random() < p else 0)
    return e

def run_shots(layers, num_shots, pi, T, p_by_state, noisy_qubit=0):
    sim = stim.TableauSimulator()
    N = len(layers)
    outcomes = []
    for _ in range(num_shots):
        sim.set_inverse_tableau_to_identity()
        e = sample_correlated_Z_errors(N, pi, T, p_by_state)
        for j in range(N):
            sim.do_circuit(layers[j])
            if e[j] == 1:
                sim.z(noisy_qubit) # inject Pauli-Z after layer j
        # read out measurement record if present:
        outcomes.append(sim.current_measurement_record())
    return outcomes
```

Notes for correctness.

N1. `TableauSimulator` operations `sim.z(q)`, `sim.x(q)`, `sim.y(q)` apply Paulis exactly (no sampling inside Stim), which is what we need for correlated noise.

N2. If your layers include measurements/resets, `current_measurement_record()` returns the shot's measurement bits in order. You can also compute detectors/observables if you build them into the circuit and use appropriate Stim APIs.

N3. The correlated noise sampling is independent of Stim and can be swapped out (e.g. burst model (10), multi-qubit subsets, etc.).

6.4 Pauli-frame subtlety (multi-qubit, general Clifford circuits)

The injection rule "apply Z after layer j" is unambiguous in the *physical frame*. In a Clifford circuit, Paulis propagate:

$$C_j(P)C_j^\dagger \in \mathcal{P}_n.$$

If your noise model is defined in a *toggling frame* or in a *gate-local frame*, you must specify where in (1) the Pauli is inserted.

This note adopts the standard interleaving convention: *noise acts after the ideal layer*. If you prefer "noise before the layer", adjust by conjugation:

$$\text{(before)} \quad P \text{ then } C_j \;\equiv\; C_j \text{ then } (C_j P C_j^\dagger).$$

In practice: keep the rule consistent, and inject Paulis at the chosen location accordingly.

7 Worked specification: two-state correlated dephasing channel

For replication, here is a complete minimal specification (single qubit):

- Hidden states: $s \in \{0, 1\}$.
- Transition matrix: T as in (7) with parameter γ.
- Initial distribution: stationary $\pi = (1/2, 1/2)$ (valid for (7)).
- Conditional channels:

$$\mathcal{E}^{(0)}(\rho) = (1 - p_{\text{low}})\rho + p_{\text{low}} Z\rho Z, \quad \mathcal{E}^{(1)}(\rho) = (1 - p_{\text{high}})\rho + p_{\text{high}} Z\rho Z.$$

A single shot:

$$s_1 \sim \pi, \quad s_{j+1} \sim T_{s_j,\cdot}, \quad e_j \sim \text{Bernoulli}(p_{s_j}).$$

Inject Z after layer j if $e_j = 1$.

8 Extensions (brief)

8.1 From pure Z noise to full Pauli noise

Replace (3) by a per-state distribution over $\{I, X, Y, Z\}$ (single qubit) or over $\mathcal{P}_n$ (multi-qubit). For single qubit:

$$\mathcal{E}^{(a)}(\rho) = q_a(I)\rho + q_a(X)X\rho X + q_a(Y)Y\rho Y + q_a(Z)Z\rho Z.$$

Temporal correlation is still fully controlled by the same hidden Markov chain.

8.2 Spatial correlations

To model spatially correlated drift, share the same s_j across many qubits (global drift), or use a small set of latent variables (e.g. one per region) that evolve Markovianly and drive local Pauli channels. This preserves Stim compatibility because the injected errors remain Pauli.

9 Critical self-check (replication checklist)

This section is written as if by an independent reader trying to reproduce the protocol; it highlights common ambiguities and resolves them.

C1. Is each layer channel a valid CPTP map? Yes. Conditioned on $s_j = a$, $\mathcal{E}^{(a)}$ is a convex mixture of unitary Paulis (3) or (4), hence CPTP.

C2. Where exactly is noise inserted relative to gates? By definition (1), noise acts *after* the ideal layer unitary C_j. If a different convention is desired (before gates), it must be implemented by conjugation as explained in the Pauli-frame note.

C3. What produces temporal correlations? Only the hidden chain s_j. If T is close to identity (large diagonals), s_j changes slowly, making adjacent layers share similar error rates. Setting T to the identity yields the quasi-static limit.

C4. Can this be simulated with Stim without modifying Stim? Yes. Stim cannot generate correlated noise internally, but it can apply deterministic Paulis between Clifford layers. We therefore sample correlated errors externally and inject them through `TableauSimulator` Pauli operations.

C5. Is the sampling procedure well-defined and efficient? Yes. Sampling s_j is $O(NK)$ (often $K \leq 2$), and sampling errors is $O(N)$ (or $O(Nn)$ for n qubits). This is negligible compared to large-shot stabilizer sampling.

C6. Does the two-state model have a well-defined stationary distribution? For (7) with symmetric flips, the stationary distribution is $(1/2, 1/2)$. For a general two-state matrix, solve $\pi = \pi T$ and use that π in (11).

C7. What exactly is "low-frequency" here? Operationally: $\gamma \ll 1$ (or T nearly diagonal), so the environment changes slowly across layers. This reproduces the "drift" intuition without simulating a continuous-time Hamiltonian.

10 Summary

A Hidden Markov Pauli channel provides a Hamiltonian-free and Stim-compatible way to model low-frequency (time-correlated) noise:

sample $s_1 \sim \pi$, $s_{j+1} \sim T_{s_j,\cdot}$; then sample Pauli errors from $\mathcal{E}^{(s_j)}$; inject Paulis between Clifford layers in S

The model is transparent, tunable (mean rate, burstiness, correlation time), and can be extended to full Pauli noise and spatial correlations while remaining within stabilizer simulation.